

Instituto Tecnológico de Estudios Superiores de Zamora



Unidad 2

Actividad 1: Definición de las operaciones de las API Rest del Proyecto

Arquitectura de Servicios

Roberto Suárez Sinzun

Diego Armando Cazarez Ramírez ISC

21010268

Vie. 27 de febrero de 2026

Índice

1. Descripción General del Sistema	3
Tecnologías de Desarrollo.....	3
Recursos del Sistema	3
Convenciones de la API	3
Códigos de Respuesta.....	3
Descripción del Recurso	4
Operaciones	4
1.1 Registrar Usuario.....	4
1.2 Login	5
1.3 Consultar Usuario por ID.....	5
1.4 Desactivar Usuario	6
1.5 Activar Usuario	6
1.6 Cambiar Rol de Usuario.....	7
Descripción del Recurso	8
Operaciones	8
2.1 Crear Categoría.....	8
2.2 Editar Categoría	8
2.3 Desactivar Categoría	9
2.4 Consultar Categorías por Usuario	10
2.5 Consultar Categorías por Estado	10
Descripción del Recurso	11
Operaciones	11
3.1 Registrar Movimiento	11
3.2 Editar Movimiento	12
3.3 Cancelar Movimiento	12
3.4 Consultar Movimiento por ID.....	13
3.5 Consultar Movimientos por Usuario y Tipo	13
3.6 Consultar Movimientos por Mes	14
3.7 Consultar Movimientos por Categoría	14
3.8 Consultar Movimientos por Tipo de Pago.....	15
Descripción del Recurso	16
Operaciones	16
4.1 Crear Planeación Mensual.....	16
4.2 Consultar Planeación por ID	17
4.3 Consultar Historial de Planeaciones por Usuario	17
4.4 Consultar Planeación por Mes y Año	18
4.5 Editar Planeación.....	18

1. Descripción General del Sistema

Billey es una aplicación de finanzas personales que permite a los usuarios registrar sus ingresos y gastos, clasificarlos por categorías y generar un análisis financiero mensual basado en ingresos reales, gastos acumulados y metas de ahorro. El enfoque principal no es un presupuesto tradicional, sino una planeación financiera mensual orientada a resultados reales.

Tecnologías de Desarrollo

Lenguaje	Python 3.11
Framework	FastAPI
Gestor de BD	MySQL 8.0
ORM	SQLAlchemy
Desarrollador	Diego Armando Cazarez Ramírez

Recursos del Sistema

Identificador	Base URL	Descripción
Recurso 1	/usuarios	Gestión de usuarios, autenticación y roles
Recurso 2	/categorias	Administración de categorías por usuario
Recurso 3	/movimientos	Registro y consulta de ingresos y gastos
Recurso 4	/planeacion	Planeación financiera mensual

Convenciones de la API

- Protocolo: HTTP/HTTPS
- Formato de datos: JSON
- Fechas en formato ISO 8601: YYYY-MM-DDTHH:MM:SS
- Montos en tipo decimal con dos decimales: 1500.00

Códigos de Respuesta

Código	Estado	Descripción
200	OK	Consulta o actualización exitosa
201	Created	Recurso creado exitosamente
400	Bad Request	Datos de entrada inválidos o regla de negocio violada
403	Forbidden	El usuario existe pero está inactivo o sin permiso
404	Not Found	Recurso no encontrado
409	Conflict	Conflicto de unicidad (correo ya registrado, planeación duplicada, etc.)
500	Internal Server Error	Error interno del servidor

Recurso 1	Usuarios	Base URL: /usuarios
--------------	----------	---------------------

Descripción del Recurso

Gestiona el registro y autenticación de los usuarios de la aplicación. Cada usuario tiene acceso exclusivo a sus propios movimientos, categorías y planeaciones.

Operaciones

1.1 Registrar Usuario

URL	/usuarios/registrar
Método HTTP	POST
Actor	Usuario no autenticado
Entrada	<pre>{ "nombre": "string", "correo": "string", "password": "string" }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	- El correo debe ser único en el sistema. - El correo debe tener formato válido (usuario@dominio.com). - La contraseña debe tener mínimo 8 caracteres. - El nombre no puede estar vacío. - El sistema asigna automáticamente: estatus = true (activo) y rol = 'usuario'. - La contraseña se almacena como hash SHA-256; nunca se guarda el texto plano. El usuario con rol 'admin' se crea directamente en la base de datos por el desarrollador durante la configuración inicial del sistema.
Excepciones	400 — El nombre está vacío o la contraseña tiene menos de 8 caracteres. 400 — El correo no tiene formato válido. 409 — El correo ya está registrado en el sistema. 500 — Error interno al insertar en la base de datos.

1.2 Login

URL	/usuarios/login
Método HTTP	POST
Actor	Usuario registrado
Entrada	<pre>{ "correo": string, "password": string }</pre>
Salida	<pre>{ "codigo": int, "id_usuario": int, "nombre": string, "estatus": boolean, "rol": string }</pre>
Reglas de Negocio	• El correo debe existir en el sistema. • La contraseña debe coincidir con el hash SHA-256 almacenado. • Solo usuarios con estatus = true (activo) pueden iniciar sesión. • En caso de credenciales incorrectas se retorna código 400.
Excepciones	• 400 — El correo no está registrado en el sistema. • 400 — La contraseña no coincide con el hash almacenado. • 403 — El usuario existe pero tiene estatus inactivo (false). • 500 — Error interno al consultar la base de datos.

1.3 Consultar Usuario por ID

URL	/usuarios/{id_usuario}
Método HTTP	GET
Actor	Usuario
Entrada	id_usuario (path param, int)
Salida	<pre>{ "id_usuario": int, "nombre": string, "correo": string, "estatus": boolean, "rol": string }</pre>
Reglas de Negocio	• Un usuario solo puede consultar su propia información. • Un usuario con rol 'admin' puede consultar la información de cualquier usuario. • No se retorna el campo password en ningún caso. • Se retornan los campos estatus y rol para reflejar el estado actual.
Excepciones	• 404 — No existe un usuario con el ID indicado. • 500 — Error interno al consultar la base de datos.

1.4 Desactivar Usuario

URL	/usuarios/{id_usuario}/desactivar
Método HTTP	PATCH
Actor	Usuario / Administrador
Entrada	id_usuario (path param, int)
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	• Se cambia el campo estatus a true. • Solo un usuario con rol 'admin' puede activar usuarios. • El usuario podrá iniciar sesión nuevamente una vez activado.
Excepciones	• 400 — El usuario ya se encuentra activo. • 404 — No existe un usuario con el ID indicado. • 500 — Error interno al actualizar la base de datos.

1.5 Activar Usuario

URL	/usuarios/{id_usuario}/activar
Método HTTP	PATCH
Actor	Administrador
Entrada	id_usuario (path param, int)
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	• Se cambia el campo estatus a true. • Solo un usuario con rol 'admin' puede activar usuarios. • El usuario podrá iniciar sesión nuevamente una vez activado.
Excepciones	• 400 — El usuario ya se encuentra activo. • 404 — No existe un usuario con el ID indicado. • 500 — Error interno al actualizar la base de datos.

1.6 Cambiar Rol de Usuario

URL	/usuarios/{id_usuario}/rol
Método HTTP	PATCH
Actor	Administrador
Entrada	<pre>{ "rol": string // "admin" "usuario" }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	• Solo se permiten los valores 'admin' o 'usuario' para el campo rol. • Solo un usuario con rol 'admin' puede cambiar el rol de otros usuarios. • No se puede cambiar el rol de un usuario inactivo.
Excepciones	• 400 — El valor de rol no es válido (diferente a 'admin' o 'usuario'). • 400 — El usuario está inactivo; no se puede cambiar su rol. • 404 — No existe un usuario con el ID indicado. • 500 — Error interno al actualizar la base de datos.

Recurso 2	Categorías	Base URL: /categorias
---------------------	-------------------	-----------------------

Descripción del Recurso

Administra las categorías utilizadas para clasificar los movimientos financieros de cada usuario. Las categorías son personales: cada usuario gestiona las suyas. Pueden desactivarse sin eliminarse para preservar el historial de movimientos asociados.

Operaciones

2.1 Crear Categoría

URL	/categorias/crear
Método HTTP	POST
Actor	Usuario
Entrada	<pre>{"id_usuario": int, "nombre": "string"}</pre>
Salida	<pre>{"codigo": 201, "mensaje": "Categoría creada exitosamente", "id_categoria": int}</pre>
Reglas de Negocio	- El nombre no puede estar vacío. - No puede existir otra categoría con el mismo nombre para el mismo usuario. - El sistema asigna automáticamente estado = true (activo) al crearse
Excepciones	400 — El nombre de la categoría está vacío. 409 — Ya existe una categoría con ese nombre para el mismo usuario. 500 — Error interno al insertar en la base de datos.

2.2 Editar Categoría

URL	/categorias/{id_categoria}/editar
Método HTTP	PUT
Actor	Usuario
Entrada	<pre>{ "nombre": string, // opcional "descripcion": string // opcional }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>

Reglas de Negocio	• La categoría debe pertenecer al usuario. • El nuevo nombre no puede estar vacío. • No puede coincidir con el nombre de otra categoría del mismo usuario. • Solo se pueden editar categorías con estado activo (true).
Excepciones	• 400 — El nombre está vacío. • 400 — La categoría está inactiva; no puede editarse. • 404 — No existe una categoría con el ID indicado. • 409 — Ya existe otra categoría con ese nombre para el mismo usuario. • 500 — Error interno al actualizar la base de datos.

2.3 Desactivar Categoría

URL	/categorias/{id_categoria}/desactivar
Método HTTP	PATCH
Actor	Usuario
Entrada	id_categoria (path param, int)
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	• La categoría debe pertenecer al usuario. • No se elimina el registro; se cambia el campo estado a false. • Una categoría desactivada no puede asignarse a nuevos movimientos. • Los movimientos históricos asociados no se ven afectados.
Excepciones	• 400 — La categoría ya está inactiva. • 404 — No existe una categoría con el ID indicado. • 500 — Error interno al actualizar la base de datos.

2.4 Consultar Categorías por Usuario

URL	/categorias/{id_usuario}
Método HTTP	GET
Actor	Usuario
Entrada	id_usuario (path param, int)
Salida	<pre>[{ "id_categoria": int, "nombre": string, "estado": boolean }, ...]</pre>
Reglas de Negocio	- Solo se retornan las categorías pertenecientes al usuario indicado. - Por defecto se retornan activas e inactivas.
Excepciones	404 — No existe un usuario con el ID indicado. 500 — Error interno al consultar la base de datos.

2.5 Consultar Categorías por Estado

URL	/categorias/{id_usuario}/estado/{estado}
Método HTTP	GET
Actor	Usuario
Entrada	usuario (path param, int) estado (path param, boolean) // true false
Salida	<pre>[{ "id_categoria": int, "nombre": string, "estado": boolean }, ...]</pre>
Reglas de Negocio	- Solo se retornan categorías del usuario indicado. - true retorna categorías activas; false retorna inactivas.
Excepciones	400 — El valor de estado no es un booleano válido. 404 — No existe un usuario con el ID indicado. 500 — Error interno al consultar la base de datos.

Recurso 3	Movimientos	Base URL: /movimientos
--------------	-------------	------------------------

Descripción del Recurso

Núcleo del sistema. Registra cada operación financiera del usuario, ya sea un ingreso o un gasto. Cada movimiento está asociado a un usuario, una categoría y una planeación mensual. Al registrarse o modificarse, actualiza automáticamente los totales de la planeación correspondiente

Operaciones

3.1 Registrar Movimiento

URL	/movimientos/registrar
Método HTTP	POST
Actor	Usuario
Entrada	<pre>{ "id_usuario": int, "id_categoria": int, "monto": decimal, "tipo_movimiento": string, // "ingreso" "gasto" "fecha": datetime, // YYYY-MM-DDTHH:MM:SS "descripcion": string, // opcional "tipo_pago": string // "efectivo" "tarjeta_debito" "tarjeta_credito" }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string, "id_movimiento": int }</pre>
Reglas de Negocio	• El monto debe ser mayor a 0. • tipo_movimiento debe ser 'ingreso' o 'gasto'. • La fecha no puede ser futura. • id_categoria debe pertenecer al usuario y estar activa. • El sistema busca automáticamente la planeación del usuario correspondiente al mes y año de la fecha del movimiento. • Si no existe una planeación para ese periodo, el sistema la crea automáticamente con ingreso_estimado = 0.00 y meta_ahorro = 0.00. • El sistema asigna automáticamente estatus = 'activo' al registrarse. • Si tipo es 'ingreso': sumar monto a ingreso_real de la planeación. • Si tipo es 'gasto': sumar monto a gasto_total de la planeación. • Recalcular ahorro_real = ingreso_real – gasto_total en la planeación.
Excepciones	• 400 — El monto es igual o menor a 0. • 400 — La fecha es futura. • 400 — tipo_movimiento no es 'ingreso' ni 'gasto'. • 400 — La categoría está inactiva o no pertenece al usuario.

	• 400 — La planeación no corresponde al mes/año de la fecha del movimiento. • 404 — La planeación o categoría indicada no existe. • 500 — Error interno al insertar en la base de datos.
--	---

3.2 Editar Movimiento

URL	/movimientos/editar/{id_movimiento}
Método HTTP	PUT
Actor	Usuario
Entrada	<pre>{ "id_categoria": int, // opcional "monto": decimal, // opcional "tipo_movimiento": string, // opcional "descripcion": string, // opcional "tipo_pago": string // opcional }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	• Solo se pueden editar movimientos con estatus 'activo'. • El movimiento debe pertenecer al usuario. • Si cambia tipo de 'ingreso' a 'gasto': restar del ingreso_real y sumar al gasto_total. • Si cambia tipo de 'gasto' a 'ingreso': restar del gasto_total y sumar al ingreso_real. • Si solo cambia el monto: ajustar la diferencia en el campo correspondiente de la planeación. Recalcular ahorro_real = ingreso_real – gasto_total en la planeación.
Excepciones	• 400 — El movimiento tiene estatus 'cancelado'; no puede editarse. • 400 — El monto indicado es igual o menor a 0. • 404 — No existe un movimiento con el ID indicado. • 500 — Error interno al actualizar la base de datos.

3.3 Cancelar Movimiento

URL	/movimientos/cancelar/{id_movimiento}
Método HTTP	PATCH
Actor	Usuario
Entrada	<pre>{ "motivo": string // opcional }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string } }</pre>

Reglas de Negocio	- No se elimina el registro; se cambia estatus a 'cancelado'. - El movimiento debe pertenecer al usuario. - Si era 'ingreso': restar su monto del ingreso_real de la planeación. - Si era 'gasto': restar su monto del gasto_total de la planeación.
Excepciones	400 — El movimiento ya tiene estatus 'cancelado'. 404 — No existe un movimiento con el ID indicado. 500 — Error interno al actualizar la base de datos.

3.4 Consultar Movimiento por ID

URL	/movimientos/{id_movimiento}
Método HTTP	GET
Actor	Usuario
Entrada	id_movimiento (path param, int)
Salida	<pre>{ "id_movimiento": int, "id_usuario": int, "id_categoria": int, "id_planeacion": int, "monto": decimal, "tipo_movimiento":string, "fecha": datetime, "descripcion": string, "tipo_pago": string, "estatus": string }</pre>
Reglas de Negocio	El movimiento debe pertenecer al usuario.
Excepciones	404 — No existe un movimiento con el ID indicado. 500 — Error interno al consultar la base de datos.

3.5 Consultar Movimientos por Usuario y Tipo

URL	/movimientos/{id_usuario}/tipo/{tipo_movimiento}
Método HTTP	GET
Actor	Usuario
Entrada	id_usuario (path param, int) tipo_movimiento (path param, string) // 'ingreso' 'gasto'
Salida	[{ movimiento }, { movimiento }, ...]
Reglas de Negocio	- Solo se retornan movimientos del usuario especificado. - Se filtra por el tipo indicado (ingreso o gasto).

	No se incluyen movimientos con estatus 'cancelado' por defecto.
Excepciones	• 400 — tipo_movimiento no es 'ingreso' ni 'gasto'. • 404 — No existe un usuario con el ID indicado. • 500 — Error interno al consultar la base de datos.

3.6 Consultar Movimientos por Mes

URL	/movimientos/{id_usuario}/mes/{mes}/anio/{anio}
Método HTTP	GET
Actor	Usuario
Entrada	<pre> usuario (path param, int) mes (path param, int) // 1-12 anio (path param, int) </pre>
Salida	[{ movimiento }, { movimiento }, ...]
Reglas de Negocio	• Se retornan todos los movimientos activos del usuario en el mes y año indicados. • El filtro se aplica sobre el id_planeacion asociado al movimiento.
Excepciones	• 400 — El mes no está entre 1 y 12. • 404 — No existe planeación para ese usuario, mes y año. • 500 — Error interno al consultar la base de datos.

3.7 Consultar Movimientos por Categoría

URL	/movimientos/{id_usuario}/categoria/{id_categoria}
Método HTTP	GET
Actor	Usuario
Entrada	<pre> id_usuario (path param, int) id_categoria (path param, int) </pre>
Salida	[{ movimiento }, { movimiento }, ...]
Reglas de Negocio	• La categoría debe pertenecer al usuario. • Se retornan todos los movimientos activos de esa categoría.
Excepciones	• 400 — La categoría no pertenece al usuario indicado. • 404 — No existe la categoría con el ID indicado. • 500 — Error interno al consultar la base de datos.

3.8 Consultar Movimientos por Tipo de Pago

URL	/movimientos/{id_usuario}/pago/{tipo_pago}
Método HTTP	GET
Actor	Usuario
Entrada	<pre>id_usuario (path param, int) tipo_pago (path param, string) // 'efectivo' 'tarjeta_debito' 'tarjeta_credito'</pre>
Salida	[{ movimiento }, { movimiento }, ...]
Reglas de Negocio	• Se retornan los movimientos activos del usuario con el tipo de pago indicado. • Útil para analizar comportamiento de gasto por método de pago.
Excepciones	• 400 — tipo_pago no corresponde a un valor válido. • 404 — No existe un usuario con el ID indicado. • 500 — Error interno al consultar la base de datos.

Recurso 4	Planeación Financiera Mensual	Base URL: /planeacion
--------------	-------------------------------	-----------------------

Descripción del Recurso

Representa el resumen financiero de un usuario para un mes y año específicos. Almacena el ingreso estimado que el usuario espera recibir, acumula el ingreso real y el gasto total conforme se registran movimientos, y calcula el ahorro real del periodo. Solo puede existir una planeación por usuario por combinación de mes y año. Las planeaciones no se eliminan ni desactivan para preservar el historial financiero.

Operaciones

4.1 Crear Planeación Mensual

URL	/planeacion/crear
Método HTTP	POST
Actor	Usuario
Entrada	<pre>{ "id_usuario": int, "mes": int, "anio": int, "ingreso_estimado": decimal, // opcional "meta_ahorro": decimal // opcional }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string, "id_planeacion": int }</pre>
Reglas de Negocio	- El mes debe estar entre 1 y 12. - El año debe ser mayor a 0. - ingreso_estimado debe ser mayor a 0 si se proporciona. - meta_ahorro debe ser mayor o igual a 0 si se proporciona. - No puede existir otra planeación para el mismo usuario, mes y año. - Si no se proporcionan, el sistema asigna ingreso_estimado = 0.00 y meta_ahorro = 0.00. - El sistema asigna automáticamente: ingreso_real = 0.00, gasto_total = 0.00, ahorro_real = 0.00. - Si ya existe una planeación creada automáticamente por el sistema para ese periodo, esta operación permite actualizarla con los valores de ingreso_estimado y meta_ahorro definidos por el usuario.
Excepciones	400 — El mes no está entre 1 y 12. 400 — ingreso_estimado es igual o menor a 0. 400 — meta_ahorro es negativa. 409 — Ya existe una planeación para ese usuario, mes y año. 500 — Error interno al insertar en la base de datos.

4.2 Consultar Planeación por ID

URL	/planeacion/{id_planeacion}
Método HTTP	GET
Actor	Usuario
Entrada	id_planeacion (path param, int)
Salida	<pre>{ "id_planeacion": int, "id_usuario": int, "mes": int, "anio": int, "ingreso_estimado": decimal, "ingreso_real": decimal, "gasto_total": decimal, "meta_ahorro": decimal, "ahorro_real": decimal }</pre>
Reglas de Negocio	La planeación debe pertenecer al usuario.
Excepciones	404 — No existe una planeación con el ID indicado. 500 — Error interno al consultar la base de datos.

4.3 Consultar Historial de Planeaciones por Usuario

URL	/planeacion/{id_usuario}/historial
Método HTTP	GET
Actor	Usuario
Entrada	id_usuario (path param, int) año (query param, int, opcional) // ?anio=2025
Salida	[{ planeacion }, { planeacion }, ...]
Reglas de Negocio	- Se retornan todas las planeaciones del usuario ordenadas por año y mes descendente. - Se puede filtrar por año agregando el parámetro opcional ?anio={anio}.
Excepciones	404 — No existe un usuario con el ID indicado. 500 — Error interno al consultar la base de datos.

4.4 Consultar Planeación por Mes y Año

URL	/planeacion/{id_usuario}/mes/{mes}/anio/{anio}
Método HTTP	GET
Actor	Usuario
Entrada	<pre>id_usuario (path param, int) mes (path param, int) // 1-12 anio (path param, int)mes: int 1-12 (path param) anio: int (path param)</pre>
Salida	<pre>{ "id_planeacion": int, { "id_planeacion": int, "id_usuario": int, "mes": int, "anio": int, "ingreso_estimado": decimal, "ingreso_real": decimal, "gasto_total": decimal, "meta_ahorro": decimal, "ahorro_real": decimal } }</pre>
Reglas de Negocio	• Se retorna la planeación única del usuario para ese mes y año. • Si no existe, se retorna código 404.
Excepciones	• 400 — El mes no está entre 1 y 12. • 404 — No existe planeación para ese usuario, mes y año. • 500 — Error interno al consultar la base de datos.

4.5 Editar Planeación

URL	/planeacion/{id_planeacion}/editar
Método HTTP	PUT
Actor	Usuario
Entrada	<pre>{ "ingreso_estimado": decimal, // opcional "meta_ahorro": decimal // opcional }</pre>
Salida	<pre>{ "codigo": int, "mensaje": string }</pre>
Reglas de Negocio	• La planeación debe pertenecer al usuario. • Solo se permite modificar ingreso_estimado y meta_ahorro manualmente. • Los campos ingreso_real, gasto_total y ahorro_real son calculados automáticamente por el sistema; no se modifican aquí. • ingreso_estimado debe ser mayor a 0 si se proporciona.

	meta_ahorro debe ser mayor o igual a 0 si se proporciona
Excepciones	• 400 — ingreso_estimado es igual o menor a 0. • 400 — meta_ahorro es negativa. • 404 — No existe una planeación con el ID indicado. • 500 — Error interno al actualizar la base de datos.